

The logo for Oracle Academy. The word "ORACLE" is in a bold, orange, sans-serif font. Below it, the word "Academy" is in a smaller, dark gray, sans-serif font. The entire logo is centered on a light gray background, which is framed by dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Programming with PL/SQL

12-2

Improving PL/SQL Performance

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Identify the benefits of the NOCOPY hint and the DETERMINISTIC clause
 - Create subprograms which use the NOCOPY hint and the DETERMINISTIC clause
 - Use Bulk Binding FORALL in a DML statement
 - Use BULK COLLECT in a SELECT or FETCH statement
 - Use the Bulk Binding RETURNING clause

Purpose

- Until now, you have learned how to write, compile, and execute PL/SQL code without thinking much about how long the execution will take
- None of the tables you use in this course contain more than a few hundred rows, so the execution is always fast
- But in real organizations, tables can contain millions or even billions of rows
- Obviously, processing two million rows takes much longer than processing twenty rows
- In this lesson you will learn some ways to speed up the processing of very large sets of data

Using the NOCOPY Hint

- In PL/SQL and most other programming languages, there are two ways to pass parameter arguments between a calling program and a called subprogram: by value and by reference
- Passing by value means that the argument values are copied from the calling program's memory to the subprogram's memory, and copied back again when the subprogram is exited
- So while the subprogram is executing, there are two copies of each argument

Using the NOCOPY Hint

- Passing by reference means that the argument values are not copied
- The two programs share a single copy of the data
- While passing by value is safer, it can use a lot of memory and execute slowly if the argument value is large
- Look at this fragment of code:

```
CREATE OR REPLACE PACKAGE emp_pkg IS
  TYPE t_emp IS TABLE OF employees%ROWTYPE
  INDEX BY BINARY_INTEGER;
  PROCEDURE emp_proc
    (p_small_arg IN NUMBER, p_big_arg OUT t_emp);
... END emp_pkg;
```

Using the NOCOPY Hint

- Suppose EMP_PKG.EMP_PROC fetches one million EMPLOYEES rows into P_BIG_ARG
- That's a lot of memory!
- And those one million rows must be copied to the calling environment at the end of the procedure's execution
- That's a lot of time



Using the NOCOPY Hint

- Maybe we should pass P_BIG_ARG by reference instead of by value

```
CREATE OR REPLACE PACKAGE emp_pkg IS
  TYPE t_emp IS TABLE OF employees%ROWTYPE
  INDEX BY BINARY_INTEGER;
  PROCEDURE emp_proc
    (p_small_arg IN NUMBER, p_big_arg OUT t_emp);
  ...
END emp_pkg;
```


Using the NOCOPY Hint

- By default, PL/SQL IN parameter arguments are passed by reference, while OUT and IN OUT arguments are passed by value
- We can change this to pass an OUT or IN OUT argument by reference, using the NOCOPY hint

```
CREATE OR REPLACE PACKAGE emp_pkg IS
  TYPE t_emp IS TABLE OF employees%ROWTYPE
  INDEX BY BINARY_INTEGER;
  PROCEDURE emp_proc
    (p_small_arg IN NUMBER, p_big_arg OUT NOCOPY t_emp);
  ...
END emp_pkg;
```

Using the NOCOPY Hint

- Notice that NOCOPY must come immediately after the parameter mode (OUT or IN OUT)
- Specify NOCOPY to instruct the database to pass an argument as fast as possible
- This clause can significantly enhance performance when passing a large value

```
CREATE OR REPLACE PACKAGE emp_pkg IS
  TYPE t_emp IS TABLE OF employees%ROWTYPE
  INDEX BY BINARY_INTEGER;
  PROCEDURE emp_proc
    (p_small_arg IN NUMBER, p_big_arg OUT NOCOPY t_emp);
  ...
END emp_pkg;
```

Function Based Indexes

- All of the Function Based Index examples have demonstrated the use of the UPPER and LOWER functions
- While these two are frequently used in Function Based Indexes, the Oracle database is not limited to just allowing those two functions in an index
- Any valid Oracle built-in function can be used in a Function-Based Index
- Also, any database function you write yourself can be used

Function Based Indexes

- There is one rule you must remember: if you are writing your own functions to use in a Function Based Index, you must include the key word DETERMINISTIC in the function header
- In mathematics, a deterministic system is a system in which no randomness is involved in the development of future states of the system
- Deterministic models therefore produce the same output for a given starting condition

A deterministic system is a system in which the output can be predicted with 100 percent certainty. When an Oracle Database encounters a deterministic function, it attempts to use previously calculated results when possible rather than re-executing the function. Specify DETERMINISTIC for a user-defined function to indicate that the function returns the same result value whenever it is invoked with the same values for its parameters. This helps the optimizer avoid redundant function calls: if a stored function was invoked previously with the same arguments the optimizer can elect to use the previous result. Using the built-in SQL function LOWER as an example, we might have the following SELECT statement:

```
SELECT last_name
FROM employees
WHERE LOWER(last_name) = 'king'
```

This SELECT statement may return hundreds of employees with the last name of "King" regardless what case was originally used to enter the employee's last name into the database. However, for the value "King," the function is only being referred to once by the optimizer, not hundreds of times for every employee with the last name of "King."

Do not specify DETERMINISTIC for a function whose result depends on the value of session variables or the state of

schema objects, because results might vary across calls.

Function Based Indexes

- When an Oracle Database encounters a deterministic function, it attempts to use previously calculated results when possible rather than re-executing the function
- Specify DETERMINISTIC for a user-defined function to indicate that the function returns the same result value whenever it is invoked with the same values for its parameters
- Do not specify DETERMINISTIC for a function whose result depends on the value of session variables or the state of schema objects, because results might vary across calls

Function Based Indexes

- If you are defining a function that will be used in a function based index, you must tell Oracle that the function is DETERMINISTIC and will return a consistent result given the same inputs
- The built-in SQL functions UPPER, LOWER, and TO_CHAR are already defined as deterministic by Oracle so this is why you can create an index on the UPPER value of a column

Function Based Indexes

- The results of another example of Function Based Indexes is shown below
- The d_events table was queried to find any events planned for the month of May

The screenshot shows the Oracle SQL Developer interface. At the top, a command window contains the following SQL query:

```
SELECT *  
FROM d_events  
WHERE TO_CHAR(event_date, 'mon') = 'may';
```

Below the command window, the 'Results' tab is active, displaying the 'Index Columns' section. This section shows a table with the following data:

Owner	Table Name	Index Name	Used in Plan	Columns	Uniqueness	Status	Index Type	Join Index
ACADEMYSPACE	D_EVENTS	D_EVE_ID_PK		ID	UNIQUE	VALID	NORMAL	NO

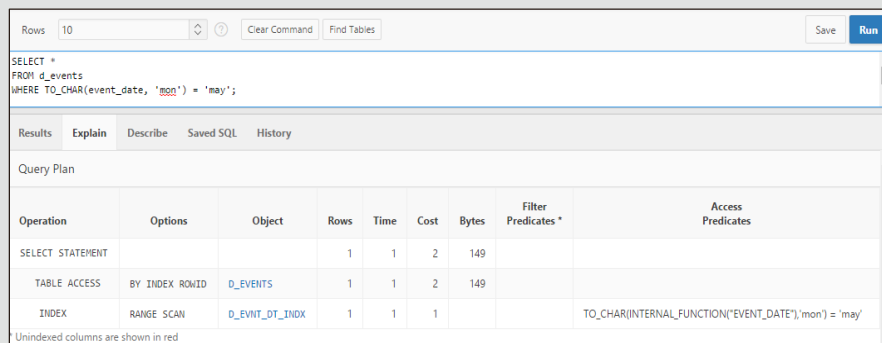
Below the 'Index Columns' section, the 'Query Plan' is displayed. It shows the following operations:

Operation	Options	Object	Rows	Time	Cost	Bytes	Filter Predicates *	Access Predicates
SELECT STATEMENT			1	1	3	141		
TABLE ACCESS	STORAGE POLL	D_EVENTS	1	1	3	141	TO_CHAR(INTERNAL_FUNCTION('EVENT_DATE')) = 'may'	TO_CHAR(INTERNAL_FUNCTION('EVENT_DATE')) = 'may'

At the bottom left of the query plan, a note states: "Unordered columns are shown in red".

Function Based Indexes

- As the Query Plan results indicate, this query executed a Full Table Scan, which can be a very time-intensive operation when a table has a lot of rows
- Even though the event_date column is indexed, the index is not used, due to the TO_CHAR expression



Rows: 10

Clear Command Find Tables Save Run

```
SELECT *  
FROM d_events  
WHERE TO_CHAR(event_date, 'mon') = 'may';
```

Results Explain Describe Saved SQL History

Query Plan

Operation	Options	Object	Rows	Time	Cost	Bytes	Filter Predicates *	Access Predicates
SELECT STATEMENT			1	1	2	149		
TABLE ACCESS	BY INDEX ROWID	D_EVENTS	1	1	2	149		
INDEX	RANGE SCAN	D_EVENT_DT_IDX	1	1	1		TO_CHAR(INTERNAL_FUNCTION('EVENT_DATE'), 'mon') = 'may'	

* Unindexed columns are shown in red

Function Based Indexes

- Once we create the following Function Based Index, we can run the same query, but this time avoid the time-intensive Full Table Scan
- The index on the event_date column can now be used

```
CREATE INDEX d_evnt_dt_indx  
ON d_events (TO_CHAR(event_date, 'mon'));
```

The screenshot shows the Oracle SQL Developer interface. At the top, there's a toolbar with 'Rows' (set to 10), 'Clear Command', 'Find Tables', 'Save', and 'Run'. Below this is a text area containing the SQL query: `SELECT *
FROM d_events
WHERE TO_CHAR(event_date, 'mon') = 'may';`. Below the query area are tabs for 'Results', 'Explain' (which is selected), 'Describe', 'Saved SQL', and 'History'. The 'Explain' tab displays a 'Query Plan' table. The table has columns: Operation, Options, Object, Rows, Time, Cost, Bytes, Filter Predicates *, and Access Predicates. The plan shows three steps: 'SELECT STATEMENT' (1 row, 1 time, 2 cost, 149 bytes), 'TABLE ACCESS BY INDEX ROWID' (1 row, 1 time, 2 cost, 149 bytes), and 'INDEX RANGE SCAN' (1 row, 1 time, 1 cost). The 'INDEX RANGE SCAN' step is highlighted in blue and points to the 'D_EVNT_DT_INDEX' object. The 'Access Predicates' column for the index step contains the text: `TO_CHAR(INTERNAL_FUNCTION('EVENT_DATE'),'mon') = 'may'`. A note at the bottom states: 'Unindexed columns are shown in red'.

Operation	Options	Object	Rows	Time	Cost	Bytes	Filter Predicates *	Access Predicates
SELECT STATEMENT			1	1	2	149		
TABLE ACCESS	BY INDEX ROWID	D_EVENTS	1	1	2	149		
INDEX	RANGE SCAN	D_EVNT_DT_INDEX	1	1	1			TO_CHAR(INTERNAL_FUNCTION('EVENT_DATE'),'mon') = 'may'

Function Based Indexes

- Now create your own PL/SQL function and try to create a Function Based Index on it:

```
CREATE OR REPLACE FUNCTION twicenum
(p_number IN NUMBER)
RETURN NUMBER IS
BEGIN
RETURN p_number * 2;
END twicenum;
```

```
CREATE INDEX emp_twicesal_idx
ON employees (twicenum(salary));
```

ORA-30553: The function is not deterministic

Using the DETERMINISTIC Clause

- If you want to create a Function Based Index on your own functions (not the built-in functions like MOD) you must create the function using the DETERMINISTIC clause:

```
CREATE OR REPLACE FUNCTION twicenum  
  (p_number IN NUMBER)  
  RETURN NUMBER DETERMINISTIC IS  
BEGIN  
  RETURN p_number * 2;  
END twicenum;
```

- Now the index can be created successfully:

```
CREATE INDEX emp_twicesal_idx  
  ON employees (twicenum(salary));
```

Using the DETERMINISTIC Clause

- Be careful!
- The word “deterministic” means that the same input value will always produce the same output value
- Look at this function:

```
CREATE OR REPLACE FUNCTION total_sal
(p_dept_id IN employees.department_id%TYPE)
RETURN NUMBER DETERMINISTIC IS
v_total_sal NUMBER;
BEGIN
  SELECT SUM(salary) INTO v_total_sal
  FROM employees WHERE department_id = p_dept_id;
  RETURN v_total_sal;
END total_sal;
```

Using the DETERMINISTIC Clause

- The function on the previous slide is not really deterministic, but the Oracle server still allowed you to create it
- What if we give everyone a salary increase?

```
UPDATE employees  
SET salary = salary * 1.10;  
COMMIT;
```

- Now the SUM(salary) values stored in the index are out-of-date, and the index will not be used unless you DROP and CREATE it again
- This will take a long time on a very large table
- Do NOT create a deterministic function which contains a SELECT statement on data which may be modified in the future

What is Bulk Binding?

- Many PL/SQL blocks contain both PL/SQL statements and SQL statements, each of which is executed by a different part of the Oracle software called the PL/SQL Engine and the SQL Engine
- A change from one engine to the other is called a context switch, which has associated overhead and takes time
- For one change, this is at most a few milliseconds
- But what if there are millions of changes?



If PL/SQL code loops through a collection performing the same DML operation for each item in the collection it is possible to reduce context switches by bulk binding the whole collection to the DML statement in one operation.

What is Bulk Binding?

- If we FETCH (in a cursor) and process millions of rows one at a time, that's millions of context switches
- And that will really slow down the execution
- FETCH is a SQL statement because it accesses database tables, but the processing is done by PL/SQL statements



What is Bulk Binding?

- Look at this code, and imagine that our EMPLOYEES table has one million rows
- How many context switches occur during one execution of the procedure?

```
CREATE OR REPLACE PROCEDURE fetch_all_emps IS
  CURSOR emp_curs IS SELECT * FROM employees;
BEGIN
  FOR v_emprec IN emp_curs LOOP
    DBMS_OUTPUT.PUT_LINE(v_emprec.first_name);
  END LOOP;
END fetch_all_emps;
```

- Remember that in a cursor FOR loop, all the fetches are still executed even though we do not explicitly code a FETCH statement

ANSWER: One million rows means one million context switches.

What is Bulk Binding?

- It would be much quicker to fetch all the rows in just one context switch within the SQL Engine
- This is what Bulk Binding does
- Of course, if all the rows are fetched in one statement, we will need an INDEX BY table of records to store all the fetched rows



What is Bulk Binding?

- If each row is (on average) 100 bytes in size, storing one million rows will need 100 megabytes of memory
- When you think about many users accessing a database, you can see how memory usage could become an issue
- So Bulk Binding is a trade-off: more memory required (possibly bad) but faster execution (good)



Bulk Binding a SELECT: Using BULK COLLECT

- Here is the one million row table from the earlier slide, this time using Bulk Binding to fetch all the rows in a single call to the SQL Engine

```
CREATE OR REPLACE PROCEDURE fetch_all_emps IS
  TYPE t_emp IS TABLE OF employees%ROWTYPE INDEX BY BINARY_INTEGER;
  v_emptab t_emp;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FOR i IN v_emptab.FIRST..v_emptab.LAST LOOP
    IF v_emptab.EXISTS(i) THEN
      DBMS_OUTPUT.PUT_LINE(v_emptab(i).last_name);
    END IF;
  END LOOP;
END fetch_all_emps;
```

- Now how many context switches are there?

ANSWER: one context switch

Bulk Binding a SELECT: Using BULK COLLECT

- When using BULK COLLECT, we do not declare a cursor because we do not fetch individual rows one at a time
- Instead, we SELECT the whole database table into the PL/SQL INDEX BY table in a single SQL statement

Bulk Binding a SELECT: Using BULK C

- Here is another example:

```
CREATE OR REPLACE PROCEDURE fetch_some_emps IS
  TYPE t_salary IS TABLE OF employees.salary%TYPE
    INDEX BY BINARY_INTEGER;
  v_salstab t_salary;
BEGIN
  SELECT salary BULK COLLECT INTO v_salstab
  FROM employees WHERE department_id = 20 ORDER BY salary;
  FOR i IN v_salstab.FIRST..v_salstab.LAST LOOP
    IF v_salstab.EXISTS(i) THEN
      DBMS_OUTPUT.PUT_LINE(v_salstab(i));
    END IF;
  END LOOP;
END fetch_some_emps;
```

Bulk Binding with DML: Using FORALL

- We may also want to speed up DML statements which process many rows
- Look at this code:

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  FOR i IN v_emptab.FIRST..v_emptab.LAST LOOP
    INSERT INTO employees VALUES v_emptab(i);
  END LOOP;
END insert_emps;
```

- Again, if we are inserting one million rows, this is one million executions of an INSERT SQL statement
- How many context switches?

ANSWER: one million context switches

Bulk Binding with DML: Using FORALL

- Just like BULK COLLECT, there is no LOOP...END LOOP code because all the rows are inserted with a single call to the SQL Engine
- The example on the slide will compile, but will not perform any inserts as the v_emptab table is not populated in this code example

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY
  BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  FORALL i IN v_emptab.FIRST..v_emptab.LAST
    INSERT INTO employees VALUES v_emptab(i);
END insert_emps;
```


Bulk Binding with DML: Using FORALL

- We can combine BULK COLLECT and FORALL
- Suppose we want to copy millions of rows from one table to another:

```
CREATE OR REPLACE PROCEDURE copy_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY
  BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST
    INSERT INTO new_employees VALUES v_emptab(i);
END copy_emps;
```

Since no columns are specified in the INSERT statement, the record structure of the collection must match the table exactly.

Bulk binds can also improve the performance when loading collections from queries. The BULK COLLECT INTO construct binds the output of the query to the collection. Populating two collections with 10,000 rows using a FOR..LOOP takes approximately 0.05 seconds. Using the BULK COLLECT INTO construct reduces this time to less than 0.01 seconds.

Bulk Binding with DML: Using FORALL

- Since no columns are specified in the INSERT statement, the record structure of the collection must match the table exactly
- Bulk binds can also improve the performance when loading collections from queries
- The BULK COLLECT INTO construct binds the output of the query to the collection. Populating two collections with 10,000 rows using a FOR..LOOP takes approximately 0.05 seconds. Using the BULK COLLECT INTO construct reduces this time to less than 0.01 seconds

Bulk Binding with DML: Using FORALL

- We can use FORALL with UPDATE and DELETE statements as well as with INSERT:

```
CREATE OR REPLACE PROCEDURE update_emps IS
  TYPE t_emp_id IS TABLE OF employees.employee_id%TYPE
    INDEX BY BINARY_INTEGER;
  v_emp_id_tab t_emp_id;
BEGIN
  SELECT employee_id BULK COLLECT INTO v_emp_id_tab FROM
employees;
  FORALL i IN v_emp_id_tab.FIRST..v_emp_id_tab.LAST
    UPDATE new_employees
      SET salary = salary * 1.05
      WHERE employee_id = v_emp_id_tab(i);
END update_emps;
```

Bulk Binding Cursor Attributes: SQL%BULK_ROWCOUNT

- In addition to implicit cursor attributes such as SQL%ROWCOUNT, Bulk Binding uses two extra cursor attributes, which are both INDEX BY tables

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE
  INDEX BY BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST
    INSERT INTO emp VALUES v_emptab(i);
  FOR i IN v_emptab.FIRST..v_emptab.LAST LOOP
    DBMS_OUTPUT.PUT_LINE('Inserted: '
      || i || ' ' || SQL%BULK_ROWCOUNT(i) || ' rows');
  END LOOP;
END insert_emps;
```

ORACLE
Academy

PLSQL 12-2
Improving PL/SQL Performance

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

35

Note the || i || inserted into the DBMS_OUTPUT statement. The %BULK_ROWCOUNT composite attribute acts like an associative array. The database deposits in the Nth element in this collection the number of rows processed by the Nth execution of FORALL. If the || i || is not included in the statement you will not see the row count increment.

Bulk Binding Cursor Attributes: SQL%BULK_ROWCOUNT

- SQL%BULK_ROWCOUNT(i) shows the number of rows processed by the ith execution of a DML statement when using FORALL:

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE
  INDEX BY BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST
    INSERT INTO emp VALUES v_emptab(i);
  FOR i IN v_emptab.FIRST..v_emptab.LAST LOOP
    DBMS_OUTPUT.PUT_LINE('Inserted: '
      || i || ' ' || SQL%BULK_ROWCOUNT(i) || ' rows');
  END LOOP;
END insert_emps;
```

Bulk Binding Cursor Attributes: SQL%BULK_EXCEPTIONS

- Look again at our first example of using FORALL:

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY
  BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST
    INSERT INTO employees VALUES v_emptab(i);
END insert_emps;
```

- What if one of the INSERTs fails, perhaps because a constraint was violated?

Bulk Binding Cursor Attributes: SQL%BULK_EXCEPTIONS

- The whole FORALL statement fails, so no rows are inserted. And you don't even know which row failed to insert!
- That has wasted a lot of time

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY
  BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST
    INSERT INTO employees VALUES v_emptab(i);
END insert_emps;
```

Bulk Binding Cursor Attributes: SQL%BULK_EXCEPTIONS

- We add SAVE EXCEPTIONS to our FORALL statement:

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY
  BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST SAVE EXCEPTIONS
    INSERT INTO employees VALUES v_emptab(i);
END insert_emps;
```



Bulk Binding Cursor Attributes: SQL%BULK_EXCEPTIONS

- Now, all the non-violating rows will be inserted
- The violating rows populate an INDEX BY table called SQL%BULK_EXCEPTIONS which has two fields: ERROR_INDEX shows which inserts failed (first, second, ...) and ERROR_CODE shows the Oracle Server predefined error code

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST SAVE EXCEPTIONS
    INSERT INTO employees VALUES v_emptab(i);
END insert_emps;
```

Bulk Binding Cursor Attributes: SQL%BULK_EXCEPTIONS

- An exception has been raised (at least one row failed to insert) so we must code the display of SQL%BULK_EXCEPTIONS in the EXCEPTION section

```
CREATE OR REPLACE PROCEDURE insert_emps IS
  TYPE t_emps IS TABLE OF employees%ROWTYPE INDEX BY BINARY_INTEGER;
  v_emptab t_emps;
BEGIN
  SELECT * BULK COLLECT INTO v_emptab FROM employees;
  FORALL i IN v_emptab.FIRST..v_emptab.LAST SAVE EXCEPTIONS
    INSERT INTO employees VALUES v_emptab(i);
EXCEPTION
  WHEN OTHERS THEN
    FOR j IN 1..SQL%BULK_EXCEPTIONS.COUNT LOOP
      DBMS_OUTPUT.PUT_LINE(SQL%BULK_EXCEPTIONS(j).ERROR_INDEX);
      DBMS_OUTPUT.PUT_LINE(SQL%BULK_EXCEPTIONS(j).ERROR_CODE);
    END LOOP;
END insert_emps;
```

ORACLE
Academy

PLSQL 12-2
Improving PL/SQL Performance

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

41

What if the goal is to proceed past any problem rows and continue processing? In order to achieve this, the SAVE EXCEPTIONS clause of the FORALL statement must be used;

The FORALL statement includes an optional SAVE EXCEPTIONS clause that allows bulk operations to save exception information and continue processing. Once the operation is complete, the exception information can be retrieved using the SQL%BULK_EXCEPTIONS attribute. This is a collection of exceptions for the most recently executed FORALL statement, with the following two fields for each exception:

SQL%BULK_EXCEPTIONS(i).ERROR_INDEX – Holds the iteration (not the subscript) of the original FORALL statement that raised the exception. In sparsely populated collections, the exception row must be found by looping through the original collection the correct number of times.

SQL%BULK_EXCEPTIONS(i).ERROR_CODE – Holds the exceptions error code. The total number of exceptions can be returned using the collections COUNT method, which returns zero if no

exceptions were raised.

Bulk Binding Cursor Attributes: SQL%BULK_EXCEPTIONS

- The FORALL statement includes an optional SAVE EXCEPTIONS clause that allows bulk operations to save exception information and continue processing
- Once the operation is complete, the exception information can be retrieved using the SQL%BULK_EXCEPTIONS attribute
- This is a collection of exceptions for the most recently executed FORALL statement, with the following two fields for each exception:
 - SQL%BULK_EXCEPTIONS(i).ERROR_INDEX
 - SQL%BULK_EXCEPTIONS(i).ERROR_CODE

SQL%BULK_EXCEPTIONS(i).ERROR_INDEX – Holds the iteration (not the subscript) of the original FORALL statement that raised the exception. In sparsely populated collections, the exception row must be found by looping through the original collection the correct number of times.

SQL%BULK_EXCEPTIONS(i).ERROR_CODE – Holds the exceptions error code. The total number of exceptions can be returned using the collections COUNT method, which returns zero if no exceptions were raised.

Using the RETURNING Clause

- Sometimes we need to DML a row, then SELECT column values from the updated row for later use:

```
CREATE OR REPLACE PROCEDURE update_one_emp
(p_emp_id    IN  employees.employee_id%TYPE,
p_salary_raise_percent IN  NUMBER) IS
v_new_salary  employees.salary%TYPE;
BEGIN
    UPDATE employees
    SET salary = salary * (1 + p_salary_raise_percent)
    WHERE employee_id = p_emp_id;
    SELECT salary INTO v_new_salary
    FROM employees
    WHERE employee_id = p_emp_id;
    DBMS_OUTPUT.PUT_LINE('New salary is: ' || v_new_salary);
END update_one_emp;
```

- Two SQL statements are required: an UPDATE and a SELECT

Using the RETURNING Clause

- However, we can do the SELECT within the UPDATE statement:

```
CREATE OR REPLACE PROCEDURE update_one_emp
  (p_emp_id    IN  employees.employee_id%TYPE,
   p_salary_raise_percent IN  NUMBER) IS
  v_new_salary  employees.salary%TYPE;
BEGIN
  UPDATE employees
  SET salary = salary * (1 + p_salary_raise_percent)
  WHERE employee_id = p_emp_id
  RETURNING salary INTO v_new_salary;
  DBMS_OUTPUT.PUT_LINE('New salary is: ' || v_new_salary);
END update_one_emp;
```

- This is faster because it makes only one call to the SQL Engine

The Oracle RETURNING clause provides the PL/SQL developer with a lot of flexibility.

The RETURNING clause to INSERTs, UPDATEs, and DELETEs helps to retrieve data from a row or rows affected by the DML statement.

If the DML operation is affecting only one row then static SQL can be used.

Using PL/SQL, the RETURNING Clause can also be used in BULK DML to return values to a collection/array for later evaluation.

The RETURNING clause eliminates inefficient network round trips and server memory usage.

Using the RETURNING Clause with FORALL

- What if we want to update millions of rows and see the updated values?

```
CREATE OR REPLACE PROCEDURE update_all_emps
  (p_salary_raise_percent IN NUMBER) IS
  TYPE t_empid IS TABLE OF employees.employee_id%TYPE
  INDEX BY BINARY_INTEGER;
  TYPE t_sal IS TABLE OF employees.salary%TYPE
  INDEX BY BINARY_INTEGER;
  v_empidtab  t_empid;
  v_saltab    t_sal;
BEGIN
  SELECT employee_id BULK COLLECT INTO v_empidtab FROM employees;
  FORALL i IN v_empidtab.FIRST..v_empidtab.LAST
  UPDATE employees
  SET salary = salary * (1 + p_salary_raise_percent)
  WHERE employee_id = v_empidtab(i);
  SELECT salary BULK COLLECT INTO v_saltab FROM employees;
END update_all_emps;
```

Using the RETURNING Clause with FORALL

- We can use RETURNING with a Bulk Binding FORALL clause:

```
CREATE OR REPLACE PROCEDURE update_all_emps
  (p_salary_raise_percent IN NUMBER) IS
  TYPE t_empid IS TABLE OF employees.employee_id%TYPE
  INDEX BY BINARY_INTEGER;
  TYPE t_sal IS TABLE OF employees.salary%TYPE
  INDEX BY BINARY_INTEGER;
  v_empidtab  t_empid;
  v_saltab    t_sal;
BEGIN
  SELECT employee_id BULK COLLECT INTO v_empidtab FROM employees;
  FORALL i IN v_empidtab.FIRST..v_empidtab.LAST
  UPDATE employees
  SET salary = salary * (1 + p_salary_raise_percent)
  WHERE employee_id = v_empidtab(i)
  RETURNING salary BULK COLLECT INTO v_saltab;
END update_all_emps;
```


Terminology

- Key terms used in this lesson included:
 - Bulk Binding
 - BULK COLLECT Clause
 - DETERMINISTIC Clause
 - FORALL
 - NOCOPY hint
 - RETURNING Clause

- Bulk Binding – Fetches all the rows in a single call to the SQL Engine.
- BULK COLLECT Clause – provides bulk processing for SELECT and FETCH statements
- DETERMINISTIC Clause – Means that the same input value will always produce the same output value and must be used to create a function-based index on your own functions.
- FORALL – provides bulk processing for DML activity
- NOCOPY hint – Passes arguments by reference rather than by value, and usually speeds up the execution of SQL statements.
- RETURNING Clause – allows the retrieval of data modified by a DML statement without triggering a separate context switch

Summary

- In this lesson, you should have learned how to:
 - Identify the benefits of the NOCOPY hint and the DETERMINISTIC clause
 - Create subprograms which use the NOCOPY hint and the DETERMINISTIC clause
 - Use Bulk Binding FORALL in a DML statement
 - Use BULK COLLECT in a SELECT or FETCH statement
 - Use the Bulk Binding RETURNING clause

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by two horizontal dark gray bars, one at the top and one at the bottom.

ORACLE

Academy